

01 Strings

Aho-Corasick

```
string matching, trie, aho-corasick
/*
@title: Aho-Corasick
@category: Strings
@complexity:  $O(N+M)$ 
@tags: string matching, trie, aho-corasick
*/
```

```
#include <bits/stdc++.h>
using namespace std;

/*
Para frecuencias
template<int E = 26> // E = Alfabeto
struct AhoCorasick {
    int nodes;
    vector<int> suf;
    vector<int> freq;
    vector<int> by_level;
    vector<bool> terminal;
    vector<array<int, E>> go;
};
```

```
AhoCorasick() {
    add_node();
    nodes = 1;
}
```

```
void add_node() {
    terminal.emplace_back();
    suf.emplace_back();
    go.emplace_back();
    freq.emplace_back();
}
```

```
void insert(string &s) {
    int pos = 0;
    for (char ch : s) {
        int c = ch - 'a';
        if (go[pos][c] == 0) {
            go[pos][c] = nodes++;
            add_node();
        }
        pos = go[pos][c];
    }
    terminal[pos] = true;
}
```

```
void build() {
    queue<int> Q;
    Q.emplace(0);
    while (not Q.empty()) {
        int u = Q.front(); Q.pop();
        by_level.emplace_back(u);
        for (int c = 0; c < E; ++c) {
            if (go[u][c]) {
                int v = go[u][c];
                Q.emplace(v);
                suf[v] = u == 0 ? 0 : go[suf[u]][c];
            }
            else {
                go[u][c] = u == 0 ? 0 : go[suf[u]][c];
            }
        }
    }
}
```

```
// Para existencia
template<int E = 26> // E = Alfabeto
struct AhoCorasick {
    int nodes;
    vector<int> suf;
    vector<int> super;
    vector<bool> terminal;
    vector<array<int, E>> go;
```

$O(N+M)$

```
AhoCorasick() {
    add_node();
    nodes = 1;
}
```

```
void add_node() {
    terminal.emplace_back();
    suf.emplace_back();
    go.emplace_back();
    super.emplace_back();
}
```

```
void insert(string &s) {
    int pos = 0;
    for (char ch : s) {
        int c = ch - 'a';
        if (go[pos][c] == 0) {
            go[pos][c] = nodes++;
            add_node();
        }
        pos = go[pos][c];
    }
    terminal[pos] = true;
}
```

```
void build() {
    queue<int> Q;
    Q.emplace(0);
    while (not Q.empty()) {
        int u = Q.front(); Q.pop();
        super[u] = (suf[u] == 0 or terminal[suf[u]] ? suf[u] : super[suf[u]]);
        for (int c = 0; c < E; ++c) {
            if (go[u][c]) {
                int v = go[u][c];
                Q.emplace(v);
                suf[v] = u == 0 ? 0 : go[suf[u]][c];
            }
            else {
                go[u][c] = u == 0 ? 0 : go[suf[u]][c];
            }
        }
    }
}
```

```
template<int E = 26>
void mark(int u, AhoCorasick<E> &AC) {
    if (u == 0) return;
    mark(AC.super[u], AC);
    if (AC.terminal[u]) {
        cout << "El nodo " << u << " ocurre como subcadena" << endl;
        AC.terminal[u] = false;
    }
    AC.super[u] = 0;
}
```

```
int main() {
    cin.tie(0) -> sync_with_stdio(false);
    AhoCorasick<26> Solver;
    vector<string> S = {"arco", "co", "barco", "oro", "toro"};
    for (string s : S) {
        Solver.insert(s);
    }
    Solver.build();
    const string T = "coroparcotoro";
    int p = 0;
    for (char ch : T) {
        int c = ch - 'a';
        p = Solver.go[p][c];
        mark(p, Solver);
    }
    /*
    for (int i = (int)Solver.by_level.size() - 1; i > 0; --i) {
        int x = Solver.by_level[i];
        Solver.freq[Solver.suf[x]] += Solver.freq[x];
    }
    for (int i = 0; i < Solver.nodes; ++i) {
        cout << "El nodo " << i << " ocurre " << Solver.freq[i] << " veces" << endl;
    }
    */
    return 0;
```

}

Manacher's Algorithm

```
palindrome, string manipulation
/*
@title: Manacher's Algorithm
@category: Strings
@complexity:  $O(N)$ 
@tags: palindrome, string manipulation
*/
```

```
vector<int> manacher_odd(string s) {
    int n = s.size();
    s = "$" + s + "-";
    vector<int> p(n + 2);
    int l = 0, r = 1;
    for(int i = 1; i <= n; i++) {
        if(i <= r) {
            p[i] = min(r - i, p[l + (r - i)]);
        }
        while(s[i - p[i]] == s[i + p[i]]) {
            p[i]++;
        }
        if(i + p[i] > r) {
            l = i - p[i], r = i + p[i];
        }
    }
    return vector<int>(begin(p) + 1, end(p) - 1);
}
```

```
vector<int> manacher(string s) {
    string t;
    for(auto c: s) {
        t += string("#") + c;
    }
    auto res = manacher_odd(t + "#");
    return vector<int>(begin(res) + 1, end(res) - 1);
}
```

Suffix Array

```
suffix array, string matching
/*
@title: Suffix Array
@category: Strings
@complexity:  $O(N \log N)$ 
@tags: suffix array, string matching
*/
```

```
#include <bits/stdc++.h>
using namespace std;
```

```
vector<int> suffix_array(string s) {
    s += char(0);
    const int n = s.size();
    vector<int> a(n);
    iota(a.begin(), a.end(), 0);
    sort(a.begin(), a.end(), [&] (int i, int j) {
        return s[i] < s[j];
    });
    vector<int> mapping(n);
    mapping[a[0]] = 0;
    for(int i = 1; i < n; i++) mapping[a[i]] = mapping[a[i - 1]] + (s[a[i - 1]] != s[a[i]]);
    int len = 1;
    vector<int> sbs(n);
    vector<int> head(n);
    vector<int> new_mapping(n);
    while(len < n) {
        for(int i = 0; i < n; i++) sbs[i] = (a[i] + n - len) % n;
        for(int i = n - 1; i >= 0; i--) head[mapping[a[i]]] = i;
        for(int i = 0; i < n; i++) {
            int x = sbs[i];
            a[head[mapping[x]]++] = x;
        }
        new_mapping[a[0]] = 0;
        for(int i = 1; i < n; i++) {
            if(mapping[a[i - 1]] != mapping[a[i]]) {
                new_mapping[a[i]] = new_mapping[a[i - 1]] + 1;
            }
        }
    }
}
```

$O(N)$

$O(N \log N)$

```
    }
    else {
        int pre = mapping[(a[i - 1] + len) % n];
        int cur = mapping[(a[i] + len) % n];
        new_mapping[a[i]] = new_mapping[a[i - 1]] + (pre != cur);
    }
}
len <= 1;
swap(mapping, new_mapping);
}
return vector<int>(a.begin() + 1, a.end()); // Ignores a[0] since it's sentinel
character.
}
```

```
vector<int> lcp_array(vector<int> &a, string s) {
    const int n = s.size();
    vector<int> rank(n);
    for(int i = 0; i < n; i++) rank[a[i]] = i;
    int k = 0;
    vector<int> lcp(n);
    for(int i = 0; i < n; i++) {
        if(rank[i] == n - 1) {
            k = 0;
            continue;
        }
        int j = a[rank[i] + 1];
        while(i + k < n and j + k < n and s[i + k] == s[j + k]) k++;
        lcp[rank[i]] = k;
        if(k) k--;
    }
    lcp.pop_back();
    return lcp;
}
```

```
int main() {
    cin.tie(0) -> sync_with_stdio(false);
    vector<int> a = suffix_array("abcbcbaba");
    vector<int> lcp = lcp_array(a, "abcbcbaba");
    for(auto x : a) cout << x << " \n"[x == a.back()];
    for(auto x : lcp) cout << x << " \n"[x == lcp.back()];
    return 0;
}
```

04 Graphs

Bellman-Ford Algorithm

shortest path, negative weights

```
/*
@title: Bellman-Ford Algorithm
@category: Graphs
@complexity:  $O(VE)$ 
@tags: shortest path, negative weights
*/
```

```
struct Edge {
    int a, b, cost;
};

int n;
vector<Edge> edges;
const int INF = 1000000000;

void solve() {
    vector<int> d(n, 0);
    vector<int> p(n, -1);
    int x;

    for (int i = 0; i < n; ++i) {
        x = -1;
        for (Edge e : edges) {
            if (d[e.a] + e.cost < d[e.b]) {
                d[e.b] = max(-INF, d[e.a] + e.cost);
                p[e.b] = e.a;
                x = e.b;
            }
        }
    }

    if (x == -1) {
        cout << "No negative cycle found.";
    } else {
        for (int i = 0; i < n; ++i)
            x = p[x];
    }
}
```

```
vector<int> cycle;
for (int v = x;; v = p[v]) {
    cycle.push_back(v);
    if (v == x && cycle.size() > 1)
        break;
}
reverse(cycle.begin(), cycle.end());

cout << "Negative cycle: ";
for (int v : cycle)
    cout << v << ' ';
cout << endl;
}
```

Eulerian Path

graph traversal, Hamiltonian path

```
/*
@title: Eulerian Path
@category: Graphs
@complexity:  $O(V + E)$ 
@tags: graph traversal, Hamiltonian path
*/
```

```
int main() {
    int n;
    vector<vector<int>> g(n, vector<int>(n));
    // reading the graph in the adjacency matrix

    vector<int> deg(n);
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j)
            deg[i] += g[i][j];
    }

    int first = 0;
    while (first < n && !deg[first])
        ++first;
    if (first == n) {
        cout << -1;
        return 0;
    }

    int v1 = -1, v2 = -1;
    bool bad = false;
    for (int i = 0; i < n; ++i) {
        if (deg[i] & 1) {
            if (v1 == -1)
                v1 = i;
            else if (v2 == -1)
                v2 = i;
            else
                bad = true;
        }
    }

    if (v1 != -1)
        ++g[v1][v2], ++g[v2][v1];

    stack<int> st;
    st.push(first);
    vector<int> res;
    while (!st.empty()) {
        int v = st.top();
        int i;
        for (i = 0; i < n; ++i)
            if (g[v][i])
                break;
        if (i == n) {
            res.push_back(v);
            st.pop();
        } else {
            --g[v][i];
            --g[i][v];
            st.push(i);
        }
    }

    if (v1 != -1) {
        for (size_t i = 0; i + 1 < res.size(); ++i) {
            if ((res[i] == v1 && res[i + 1] == v2) ||
                (res[i] == v2 && res[i + 1] == v1)) {
                vector<int> res2;
```

$O(V + E)$

```
        for (size_t j = i + 1; j < res.size(); ++j)
            res2.push_back(res[j]);
        for (size_t j = 1; j <= i; ++j)
            res2.push_back(res[j]);
        res = res2;
        break;
    }
}

for (int i = 0; i < n; ++i) {
    for (int j = 0; j < n; ++j) {
        if (g[i][j])
            bad = true;
    }
}

if (bad) {
    cout << -1;
} else {
    for (int x : res)
        cout << x << " ";
}
}
```

02 Math

Chinese Remainder Theorem

$O(N)$

modular arithmetic, number theory

```
/*
@title: Chinese Remainder Theorem
@category: Math
@complexity:  $O(N)$ 
@tags: modular arithmetic, number theory
*/
```

```
struct Congruence {
    long long a, m;
};

long long chinese_remainder_theorem(vector<Congruence> const& congruences) {
    long long M = 1;
    for (auto const& congruence : congruences) {
        M *= congruence.m;
    }

    long long solution = 0;
    for (auto const& congruence : congruences) {
        long long a_i = congruence.a;
        long long M_i = M / congruence.m;
        long long N_i = mod_inv(M_i, congruence.m);
        solution = (solution + a_i * M_i % M * N_i) % M;
    }
    return solution;
}
```

Fast Fourier Transform

$O(N \log N)$

polynomial multiplication, FFT

```
/*
@title: Fast Fourier Transform
@category: Math
@complexity:  $O(N \log N)$ 
@tags: polynomial multiplication, FFT
*/
```

```
using cd = complex<double>;
const double PI = acos(-1);

void fft(vector<cd> &a, bool invert) {
    int n = a.size();

    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1)
            j ^= bit;
        j ^= bit;

        if (i < j)
            swap(a[i], a[j]);
    }

    for (int len = 2; len <= n; len <= 1) {
```

```
double ang = 2 * PI / len * (invert ? -1 : 1);
cd wlen(cos(ang), sin(ang));
for (int i = 0; i < n; i += len) {
    cd w(1);
    for (int j = 0; j < len / 2; j++) {
        cd u = a[i+j], v = a[i+j+len/2] * w;
        a[i+j] = u + v;
        a[i+j+len/2] = u - v;
        w *= wlen;
    }
}

if (invert) {
    for (cd & x : a)
        x /= n;
}
```

Gaussian Elimination

```
linear algebra, system of equations
/*
@title: Gaussian Elimination
@category: Math
@complexity: O(N^3)
@tags: linear algebra, system of equations
*/
```

```
const double EPS = 1e-9;
const int INF = 2; // it doesn't actually have to be infinity or a big number
```

```
int gauss (vector < vector<double> > a, vector<double> & ans) {
    int n = (int) a.size();
    int m = (int) a[0].size() - 1;

    vector<int> where (m, -1);
    for (int col=0, row=0; col<m && row<n; ++col) {
        int sel = row;
        for (int i=row; i<n; ++i)
            if (abs (a[i][col]) > abs (a[sel][col]))
                sel = i;
        if (abs (a[sel][col]) < EPS)
            continue;
        for (int i=col; i<=m; ++i)
            swap (a[sel][i], a[row][i]);
        where[col] = row;

        for (int i=0; i<n; ++i)
            if (i != row) {
                double c = a[i][col] / a[row][col];
                for (int j=col; j<=m; ++j)
                    a[i][j] -= a[row][j] * c;
            }
        ++row;
    }

    ans.assign (m, 0);
    for (int i=0; i<m; ++i)
        if (where[i] != -1)
            ans[i] = a[where[i]][m] / a[where[i]][i];
    for (int i=0; i<n; ++i) {
        double sum = 0;
        for (int j=0; j<m; ++j)
            sum += ans[j] * a[i][j];
        if (abs (sum - a[i][m]) > EPS)
            return 0;
    }

    for (int i=0; i<m; ++i)
        if (where[i] == -1)
            return INF;
    return 1;
}
```

Number Theoretic Transform

```
polynomial multiplication, NTT
/*
@title: Number Theoretic Transform
@category: Math
@complexity: O(N log N)
@tags: polynomial multiplication, NTT
*/
```

O(N^3)

```
const int mod = 7340033;
const int root = 5;
const int root_1 = 4404020;
const int root_pw = 1 << 20;
```

```
void fft(vector<int> & a, bool invert) {
    int n = a.size();

    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1)
            j ^= bit;
        j ^= bit;

        if (i < j)
            swap(a[i], a[j]);
    }

    for (int len = 2; len <= n; len <= 1) {
        int wlen = invert ? root_1 : root;
        for (int i = len; i < root_pw; i <= 1)
            wlen = (int)(1LL * wlen * wlen % mod);

        for (int i = 0; i < n; i += len) {
            int w = 1;
            for (int j = 0; j < len / 2; j++) {
                int u = a[i+j], v = (int)(1LL * a[i+j+len/2] * w % mod);
                a[i+j] = u + v < mod ? u + v : u + v - mod;
                a[i+j+len/2] = u - v >= 0 ? u - v : u - v + mod;
                w = (int)(1LL * w * wlen % mod);
            }
        }

        if (invert) {
            int n_1 = inverse(n, mod);
            for (int & x : a)
                x = (int)(1LL * x * n_1 % mod);
        }
    }
}
```

XOR Basis

```
linear algebra, system of equations
/*
@title: XOR Basis
@category: Math
@complexity: O(N log A)
@tags: linear algebra, system of equations
*/
```

```
#include <bits/stdc++.h>
using namespace std;
```

```
typedef long long ll;
```

```
ll rankGauss(vector<ll> x, ll y){
    vector<ll> basis(62,0);
    int r=0;
    int n=x.size();
    for(int i=0;i<n;i++){
        for(int bit=60;bit>=0;bit--){
            if((x[i]>>bit)&1){
                if(!basis[bit]){
                    basis[bit]=x[i];
                    r++;
                    break;
                }
                x[i]^=basis[bit];
            }
        }
    }
}
```

```
ll ty=y;
for(int bit=60;bit>=0;bit--){
    if((ty>>bit)&1){
        if(basis[bit]){
            ty^=basis[bit];
        }
    }
}
if(ty!=0) return -1;
```

// The number of subset that forms y is 2^(n-r) where r is the rank of the basis and n is the number of elements in the original set.

O(N log A)

```
return r;
}
```

05 Flows

Dinic’s Algorithm $O(V^2 \cdot E)$, $O(E \cdot \sqrt{V})$ for bipartite graphs

```
max flow, residual graph
/*
@title: Dinic’s Algorithm
@category: Graphs
@complexity: O(V^2 * E), O(E * sqrt(V)) for bipartite graphs
@tags: max flow, residual graph
*/
```

```
struct FlowEdge {
    int v, u;
    long long cap, flow = 0;
    FlowEdge(int v, int u, long long cap) : v(v), u(u), cap(cap) {}
};
```

```
struct Dinic {
    const long long flow_inf = 1e18;
    vector<FlowEdge> edges;
    vector<vector<int>>> adj;
    int n, m = 0;
    int s, t;
    vector<int> level, ptr;
    queue<int> q;
```

```
Dinic(int n, int s, int t) : n(n), s(s), t(t) {
    adj.resize(n);
    level.resize(n);
    ptr.resize(n);
}
```

```
void add_edge(int v, int u, long long cap) {
    edges.emplace_back(v, u, cap);
    edges.emplace_back(u, v, 0);
    adj[v].push_back(m);
    adj[u].push_back(m + 1);
    m += 2;
}
```

```
bool bfs() {
    while (!q.empty()) {
        int v = q.front();
        q.pop();
        for (int id : adj[v]) {
            if (edges[id].cap == edges[id].flow)
                continue;
            if (level[edges[id].u] != -1)
                continue;
            level[edges[id].u] = level[v] + 1;
            q.push(edges[id].u);
        }
    }
    return level[t] != -1;
}
```

```
long long dfs(int v, long long pushed) {
    if (pushed == 0)
        return 0;
    if (v == t)
        return pushed;
    for (int& cid = ptr[v]; cid < (int)adj[v].size(); cid++) {
        int id = adj[v][cid];
        int u = edges[id].u;
        if (level[v] + 1 != level[u])
            continue;
        long long tr = dfs(u, min(pushed, edges[id].cap - edges[id].flow));
        if (tr == 0)
            continue;
        edges[id].flow += tr;
        edges[id ^ 1].flow -= tr;
        return tr;
    }
    return 0;
}
```

```
long long flow() {
    long long f = 0;
    while (true) {
        bfs();
        while (true) {
            long long t = dfs(s, flow_inf);
            if (t == 0)
                break;
            f += t;
        }
    }
    return f;
}
```

```

        fill(level.begin(), level.end(), -1);
        level[s] = 0;
        q.push(s);
        if (!bfs())
            break;
        fill(ptr.begin(), ptr.end(), 0);
        while (long long pushed = dfs(s, flow_inf)) {
            f += pushed;
        }
    }
    return f;
}
};

```

03 Trees

Heavy Light Decomposition

```

HLD, path queries, segment tree
/*
@title: Heavy Light Decomposition
@category: Trees
@complexity: O(log^2 N)
@tags: HLD, path queries, segment tree
*/

```

```

#include <bits/stdc++.h>
using namespace std;

const int N = 5 * 100000 + 1;

```

O(log^2 N)

```

int n;
int q;
int h[N];
int sz[N];
int par[N];
int nxt[N];
vector<int> G[N];

void DFS(int u) {
    sz[u] = 1;
    for (int i = 0; i < G[u].size(); ++i) {
        int v = G[u][i];
        DFS(v);
        sz[u] += sz[v];
        if (sz[v] > sz[G[u][0]]) {
            swap(G[u][i], G[u][0]);
        }
    }
}

void HLD(int u) {
    for (int v : G[u]) {
        h[v] = h[u] + 1;
        par[v] = u;
        nxt[v] = (v == G[u][0] ? nxt[u] : v);
        HLD(v);
    }
}

```

```

int lca(int u, int v) {
    while (nxt[u] != nxt[v]) {
        if (h[nxt[u]] > h[nxt[v]]) {
            u = par[nxt[u]];
        }
        else {
            v = par[nxt[v]];
        }
    }
    return h[u] < h[v] ? u : v;
}

```

```

int main() {
    cin.tie(0) -> sync_with_stdio(false);
    cin >> n >> q;
    for (int i = 1; i < n; ++i) {
        int p;
        cin >> p;
        G[p].emplace_back(i);
    }
    DFS(0);
    nxt[0] = 0;
    HLD(0);
    while (q--) {
        int u, v;
        cin >> u >> v;
        cout << lca(u, v) << '\n';
    }
    return 0;
}

```